Accelerating Deep Learning Algorithms on Memristors

Addressing Reliability Issues and Hardware-aware Pruning

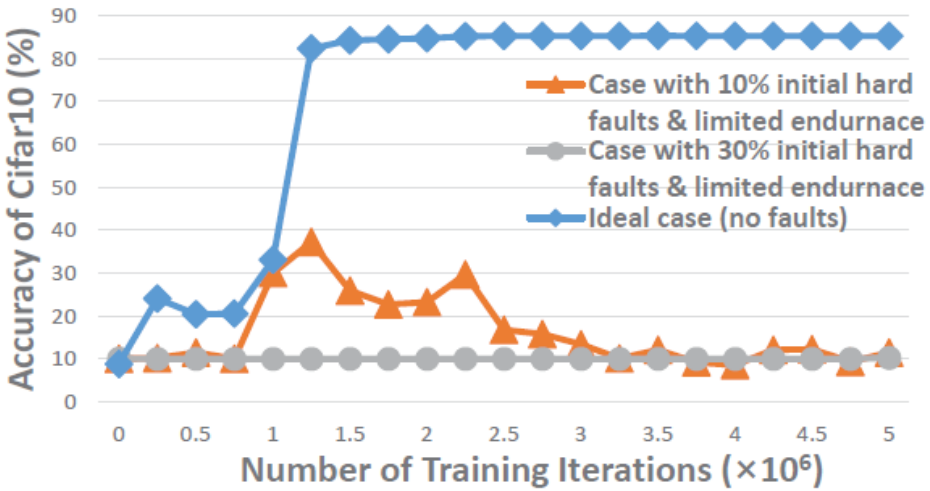
Sparsh Mittal

Background on reliability issues

- **Process variation:** deviation in parameters from their nominal values
- **Hard error:** a cell is stuck at the value 0 or 1, which happens when the write endurance limit of a cell has been reached
- **Resistance drift:** change in the resistance of the cell over time which leads to a soft-error

Reliability issues

- High defect rate & process variation =>reliability issues
- Especially for large neuromorphic systems, a ReRAM implementation leads to heavy wire congestion and poor reliability of read/write operations due to voltage-drop
- With increasing ReRAM failure rate, accuracy of NN reduces drastically



Xia et al. DAC'17

Challenges in achieving high accuracy & perf.

- Compared to SRAM, ReRAM has high write energy & latency which increases overall power consumption
- During NN training, precise tuning of ReRAM requires frequent update of weights and large number of training iterations for convergence
- This leads to high number of writes and large energy consumption
- Increasing number of training iterations leads to energy penalty and aggravates ReRAM endurance issues
- Retraining may not be sufficient in case of high fault rate

Techniques for Mitigating Hard faults (stuck-at-0 or stuck-at-1)

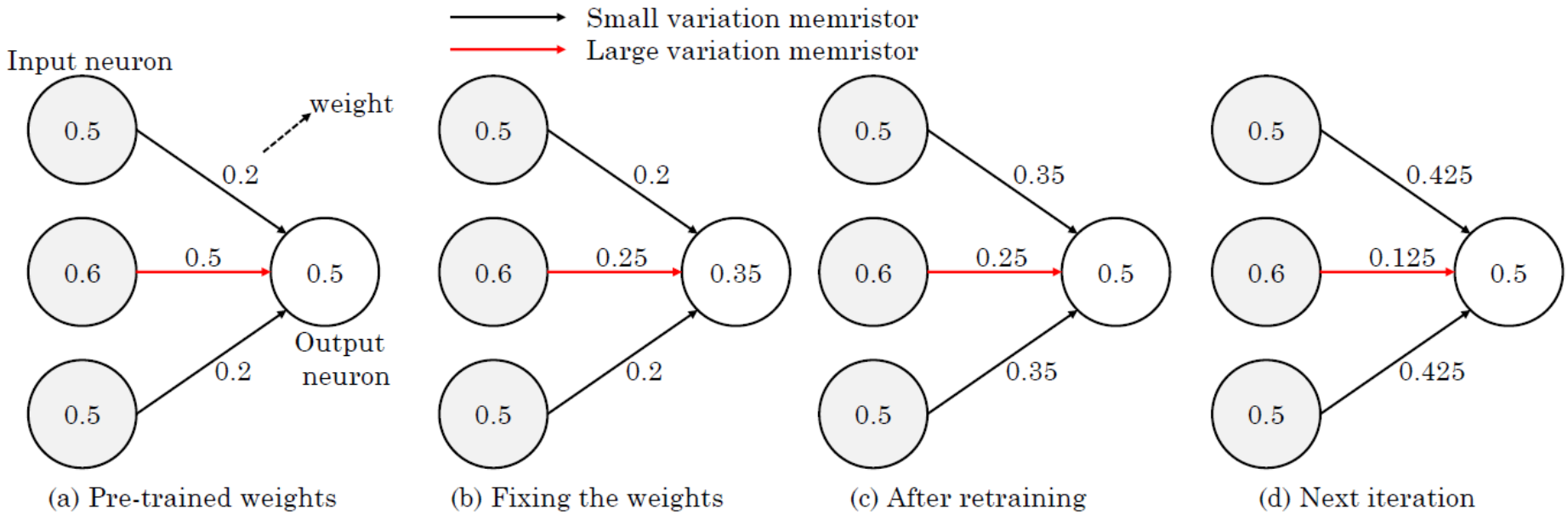
Using NN retraining (1 of 2)

- Challenge: large weight may get mapped to a ReRAM with high variation
- **Observations**: In NNs, loss in accuracy due to removal of some weights is automatically compensated during training by changing the neighboring weights (called self-healing capability)

- Technique: Train NN based on distribution of stuck-at-faults and resistance variation in crossbar.
- Lower the magnitude of weight mapped to a memristor with large variation/fault-rate

Chen et al. DATE'17

Using NN retraining (2 of 2)



This technique can remove nearly all the loss in NN accuracy due to faults/variations in ReRAM

Chen et al. DATE'17

Thresholding and Remapping

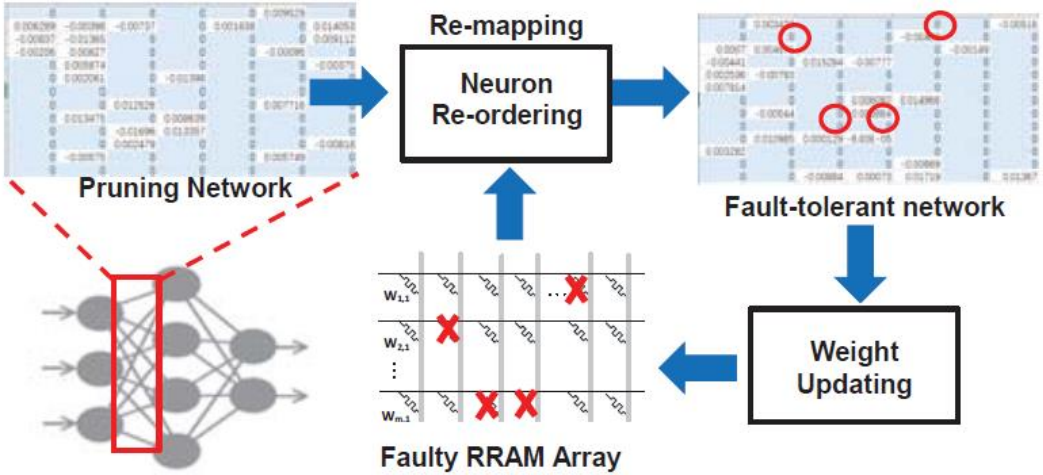
- **Observation 1:** Small weights make little contribution to the accuracy, but reduce ReRAM lifetime significantly

- **Idea 1:** In back-propagation, flush weights smaller than a threshold to zero
- This strategy boosts ReRAM lifetime while bringing only small increase in the number of training iterations

Xia et al. DAC'17

Thresholding and Remapping

- **Observation 2:** Pruning leads to a large fraction of weights becoming zero
- **Idea 2:** Use this to tolerate stuck-at-0 faults.
- For this, reorder rows/columns of the weight matrix so as to map zero weights to cells having stuck-at-0 faults



Xia et al. DAC'17

Weight significance based training

- Classify the weights into crucial and non-crucial based on their impact on final accuracy
- Perform retraining for recovering the accuracy loss due to faults: Identify stuck weights and retune remaining weights for recovering accuracy.
- Instead of giving random values to weight matrices, use pre-trained weight matrix for initializing and accelerating re-training.
- If error rate too high, use redundant columns. Remap only crucial weights to them.
- Result: By remapping only few weights, large improvement in accuracy is achieved.

Liu et al. DAC'17

Techniques for Mitigating Resistance Drift

Adaptive tuning of memristor (1 of 2)

- Challenge: Memristor gradually drifts from its original programmed state,
- e.g., a voltage of 0.1 V leads to 2% deviation from the original state in 1 s.
- => Precision of an MCA-based system decreases gradually
- Idea: Tune memristor to a specific state to compensate this effect
- Challenge: Memristor tuning takes much more latency than a regular operation (5 ms vs. 50 ns)
- => frequent calibration incurs large overhead

Li et al. DATE'14

Adaptive tuning of memristor (2 of 2)

- Idea: Observe error values over time.
- Fit an Nth-degree polynomial to show relationship between time and computation error
- Predict error value at next time instance.
- If this is less than the maximum tolerable error, skip calibration.
- Otherwise, predict the time instance at which the predicted error would have reached maximum tolerable error. Then, perform calibration at this time instance
- Result: tuning requirement reduced.

Li et al. DATE'14

MCA-configuration aware pruning
and
using suitable MCA-configuration for a pruned network

MCA-aware pruning strategy (1 of 2)

- Challenge: Pruning techniques lead to a network with irregular connections, which may not match to MCA design
- => inefficient hardware implementation

Input	Connectivity map			
I1	1	1	1	1
I2	1	1	1	1
I3	1	1	1	1
I4	1	1	1	1
Output	O1	O2	O3	O4

Original network

0	1	0	1
1	0	1	0
0	1	0	1
1	0	1	0

Pruned network

Here, 1/0 shows presence/absence of connection and not the actual weight

Ankit et al. Arxiv'17

MCA-aware pruning strategy (2 of 2)

- Idea: Prune at the granularity of MCA and not weights

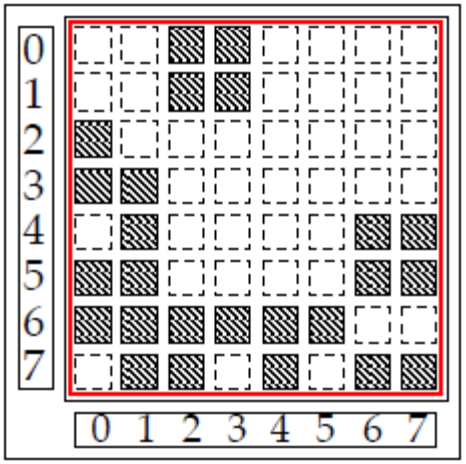
1	1	0	0
1	1	0	0
0	0	1	1
0	0	1	1

- Helps in retaining the gains of algorithmic pruning and network sparsity in memristor-based NN-engines
- Their technique is applicable to FC layers, which show no data reuse and have most of the weights of the network

Ankit et al. Arxiv'17

Impact of MCA size (1 of 2)

- MCA size decides tradeoff between parallelism/speedup/cost and utilization
- Consider an 8X8 adjacency matrix.
- Shadowed square = an edge between two vertices
- Blank square = no edge between two vertices

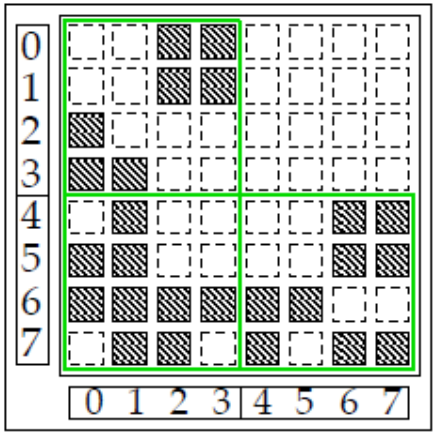


(a) 8-8 Mapping

On using 8*8 MCA, utilization is only 39.1% (=25/64)

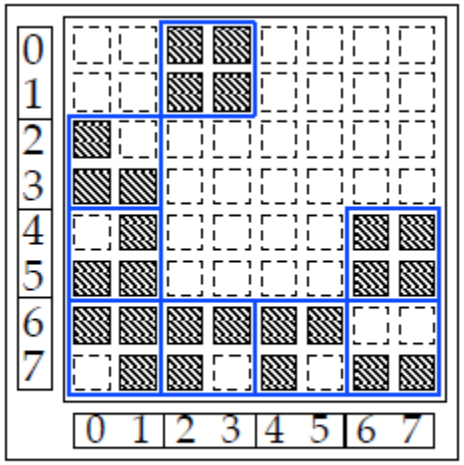
Impact of MCA size (2 of 2)

- With 4*4 MCA, utilization is 52.1% (=25/48) since subgraph in upper-right corner need not be mapped to MCA.



(b) 4-4 Mapping

- With 2*2 MCA, utilization is increased to 78.1% (=25/32)



(c) 2-2 Mapping

Song et al. HPCA'18

Solved question

- In a 6*6 adjacency matrix, following weights are zero : (0, 1), (0,5), (1, 3), (4,4), (2,0), (5,5), (4,1), (4,5), (5,2), (5,4).
- Note that index is shown as (x,y), where x and y each go from 0 to 5.

	zero				zero
			zero		
zero					
	zero			zero	zero
		zero		zero	zero

Utilization on using 3*3 MCA is

	zero				zero
			zero		
zero					
	zero			zero	zero
		zero		zero	zero

Answer: 26/36

Utilization on using 2*2 MCA is

	zero				zero
			zero		
zero					
	zero			zero	zero
		zero		zero	zero

Answer: 26/32

References

- S. Mittal, “A Survey of ReRAM-based Architectures for Processing-in-memory and Neural Networks”, Machine learning and knowledge extraction, 2018 (pdf)